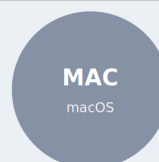
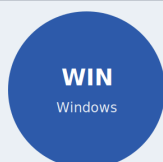




Node.js — Projeto Guiado

```
ambiente@profissional ~ node --version  
  
$ nvm install --lts  
$ npm install -g typescript ts-node  
$ docker compose up -d  
$ git commit -m "ambiente pronto"
```



Módulo 0 — Montando o Ambiente Profissional Node.js

MÓDULO 0 — MONTANDO O AMBIENTE PROFISSIONAL DE DESENVOLVIMENTO

Instalação e configuração em Windows, Linux e macOS

Joinville - 2026

Prof. Carlos Uchôa e Prof. Sestito

SUMÁRIO

1 INTRODUÇÃO	3
2 OBJETIVOS DO MÓDULO	3
3 POR QUE USAR UM GERENCIADOR DE VERSÕES DO NODE.JS	3
4 INSTALANDO O AMBIENTE NO WINDOWS	11
4.1 Ferramentas nativas da Microsoft utilizadas	4
4.2 Instalando o nvm-windows via winget	4
4.3 Instalando e ativando o Node.js LTS	5
4.4 Fixando a versão do Node.js por projeto	5
5 INSTALANDO O AMBIENTE NO LINUX	6
6 INSTALANDO O AMBIENTE NO MACOS	6
7 NPM E UTILITÁRIOS GLOBAIS	7
8 EDITOR DE CÓDIGO	8
9 GIT E GITHUB	8
9.1 Instalação	8
9.2 Configuração inicial	9
9.3 Autenticação via SSH com o GitHub	9
10 DOCKER E DOCKER COMPOSE	10
10.1 Windows 11: Docker Desktop com backend Hyper-V	10
10.2 macOS: Docker Desktop	10
10.3 Linux: Docker Engine e Compose	10
10.4 Verificação e primeiro compose do curso	11
11 CLIENTE HTTP E CLIENTE DE BANCO DE DADOS	11
12 QUALIDADE DE CÓDIGO: ESLINT, PRETTIER E EDITORCONFIG	12
13 VALIDAÇÃO FINAL: HELLO WORLD PONTA A PONTA	12
13.1 Execução local	12
13.2 Execução em container	12
14 RECURSOS ADICIONAIS DO ECOSISTEMA NODE.JS	13
14.1 Frameworks web (NestJS, Express, Fastify)	13
14.2 Acesso a dados (Prisma, TypeORM, Drizzle)	14
14.3 Validação de dados (Zod, class-validator)	14
14.4 Testes automatizados (Jest, Vitest, Supertest)	14
14.5 Observabilidade (Pino, Winston)	14
14.6 Processos em produção (PM2)	15
14.7 Documentação de API (Swagger/OpenAPI)	15
14.8 Filas e processamento assíncrono (BullMQ)	15
14.9 Autenticação e autorização (Passport.js, JWT)	15
14.10 Segurança (Helmet, rate limiting)	15
14.11 Automação de qualidade (Husky, lint-staged, Commitlint)	16
14.12 Monorepositórios (Nx, Turborepo, pnpm workspaces)	16
15 SOLUÇÃO DE PROBLEMAS	16
15.1 Windows	16
15.2 Linux	17
15.3 macOS	17
16 CONSIDERAÇÕES FINAIS	17
17 ATIVIDADES DE FIXAÇÃO	17
17.1 Atividade teórica — pesquisa e fundamentação	17
17.2 Atividade prática — hello-node profissional	18
REFERÊNCIAS	18

1 INTRODUÇÃO

Este material abre o curso profissional de Node.js e cobre a primeira etapa de qualquer projeto real: montar um ambiente de desenvolvimento consistente, reproduzível e alinhado às práticas usadas em times de tecnologia no mercado. O foco principal das instruções é o Windows 11 — sistema operacional de partida da maioria dos alunos da turma —, utilizando comandos e recursos nativos da própria Microsoft (PowerShell, winget, OpenSSH, Hyper-V), sem depender de subsistemas de terceiros. O mesmo roteiro é apresentado também para Linux e macOS, já que qualquer desenvolvedor profissional deve saber operar em mais de um sistema.

A stack padronizada para todo o curso é Node.js com TypeScript, gerenciador de pacotes npm e serviços de apoio (banco de dados, cache) executados via Docker e Docker Compose desde a primeira aula. Essa escolha não é arbitrária: reflete o que se espera de um desenvolvedor Node.js pleno/sênior no mercado de trabalho atual.

2 OBJETIVOS DO MÓDULO

Ao final deste módulo, o ambiente do aluno deve conter os itens do checklist a seguir.

Checklist do Módulo 0

- Gerenciador de versões do Node (nvm)
- npm + utilitários globais
- Editor de código configurado
- Git + SSH + GitHub
- Docker e Docker Compose
- Cliente HTTP e cliente de banco
- ESLint + Prettier + EditorConfig
- Prova real: Hello World local e em container

Cada um dos itens do checklist é detalhado em uma seção própria deste material, na ordem em que devem ser instalados.

3 POR QUE USAR UM GERENCIADOR DE VERSÕES DO NODE.JS

Em qualquer empresa é comum encontrar projetos legados presos em uma versão antiga do Node.js ao lado de projetos novos usando a versão LTS (Long-Term Support) mais recente. Se o Node.js for instalado diretamente pelo instalador oficial do site, trocar de versão exige desinstalar e reinstalar o ambiente inteiro — um processo lento e propenso a erro.

Um gerenciador de versões (nvm, fnm ou Volta) resolve esse problema com um único comando, permitindo alternar entre versões diferentes de Node.js na mesma máquina em segundos. Por esse motivo, este curso trata o uso de um gerenciador de versões como prática obrigatória, não como opcional.

Boa prática de mercado

Nunca fixe um número de versão específico do Node.js. Utilize sempre o comando "nvm ls-remote --lts" (ou equivalente) para descobrir, no dia da aula, qual é a versão LTS ativa mais recente disponível.

4 INSTALANDO O AMBIENTE NO WINDOWS 11

Todo o roteiro deste módulo para Windows utiliza exclusivamente ferramentas e comandos nativos do Windows 11, sem depender de subsistemas de terceiros. É o mesmo tipo de ambiente que um desenvolvedor encontraria em uma máquina corporativa padrão, com Windows puro.

4.1 FERRAMENTAS NATIVAS DA MICROSOFT UTILIZADAS

Antes de instalar o Node.js, é importante conhecer os recursos nativos do Windows que sustentam todo o restante do módulo:

- **winget (Windows Package Manager)** — gerenciador de pacotes oficial da Microsoft, já incluso no Windows 11 (App Installer). Instala e atualiza programas via linha de comando, sem precisar baixar instaladores manualmente.
- **PowerShell 7** — Shell de linha de comando moderno da Microsoft, multiplataforma, usado em todos os comandos deste módulo.

- **Windows Terminal** — aplicativo de terminal oficial da Microsoft, com abas, temas e suporte a múltiplos shells na mesma janela. Disponível na Microsoft Store ou via winget.
- **Cliente OpenSSH do Windows** — o Windows 11 já inclui nativamente o cliente OpenSSH (ssh, ssh-keygen, ssh-agent), usado na Seção 9.3 para autenticação com o GitHub.
- **Hyper-V** — tecnologia de virtualização nativa da Microsoft, usada como backend do Docker Desktop neste curso (Seção 10.1).

4.2 INSTALANDO O NVM-WINDOWS VIA WINGET

O curso utiliza o nvm-windows, um gerenciador de versões do Node.js que funciona nativamente no Windows, sem qualquer dependência externa.

1. Abrir o Windows Terminal ou o PowerShell (não é necessário abrir como Administrador).
2. Instalar o nvm-windows via winget:

```
winget install CoreyButler.NVMforWindows
```

3. Fechar e reabrir o terminal para que o PATH seja atualizado.

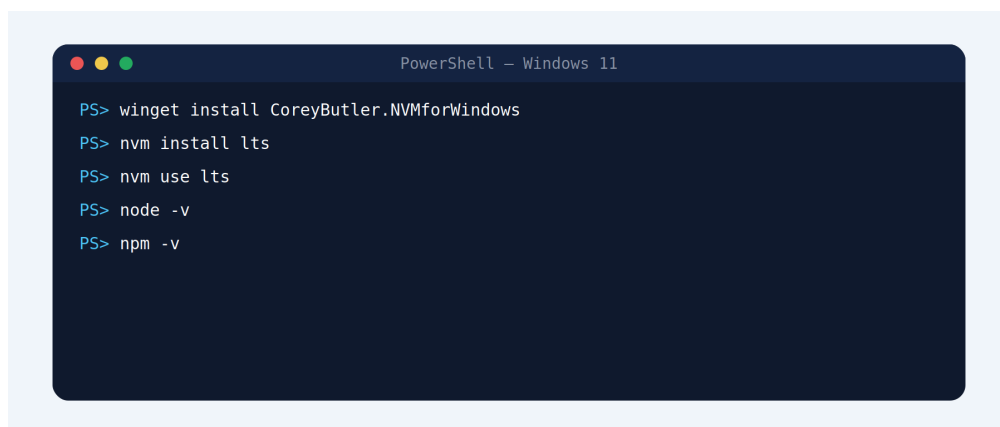


Figura 1 — Instalação do nvm-windows e do Node.js LTS no PowerShell, nativamente no Windows 11.

4.3 INSTALANDO E ATIVANDO O NODE.JS LTS

1. Instalar a versão LTS mais recente do Node.js:

```
nvm install lts
```

2. Ativar essa versão como a versão corrente:

```
nvm use lts
```

3. Verificar a instalação:

```
node -v  
npm -v
```

4.4 FIXANDO A VERSÃO DO NODE.JS POR PROJETO

Em todo repositório do curso, será criado um arquivo ".nvmrc" na raiz do projeto, contendo apenas:

```
lts/*
```

Assim, qualquer colega que clonar o projeto executa apenas "nvm use" e recebe automaticamente a versão correta do Node.js — sem precisar perguntar no chat da equipe qual versão está sendo usada.

5 INSTALANDO O AMBIENTE NO LINUX

As instruções a seguir valem para qualquer distribuição Linux baseada em Debian/Ubuntu, usada nativamente pelo aluno.

1. Instalar o nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

Verifique sempre, antes da aula, qual é a tag de instalação mais recente no repositório oficial (github.com/nvm-sh/nvm) — o número da tag no comando acima muda com o tempo.

2. Recarregar o shell:

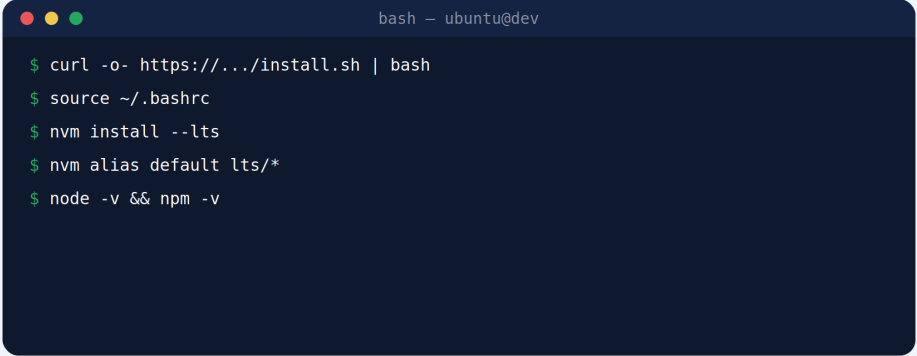
```
source ~/.bashrc
```

3. Instalar e definir a versão LTS como padrão:

```
nvm install --lts  
nvm use --lts  
nvm alias default lts/*
```

4. Verificar a instalação:

```
node -v  
npm -v
```



```
bash - ubuntu@dev
$ curl -o- https://.../install.sh | bash
$ source ~/.bashrc
$ nvm install --lts
$ nvm alias default lts/*
$ node -v && npm -v
```

Figura 2 — Instalação do nvm e do Node.js LTS em ambiente Linux.

6 INSTALANDO O AMBIENTE NO MACOS

Pré-requisito: Homebrew instalado (disponível em brew.sh).

1. Instalar o nvm via Homebrew:

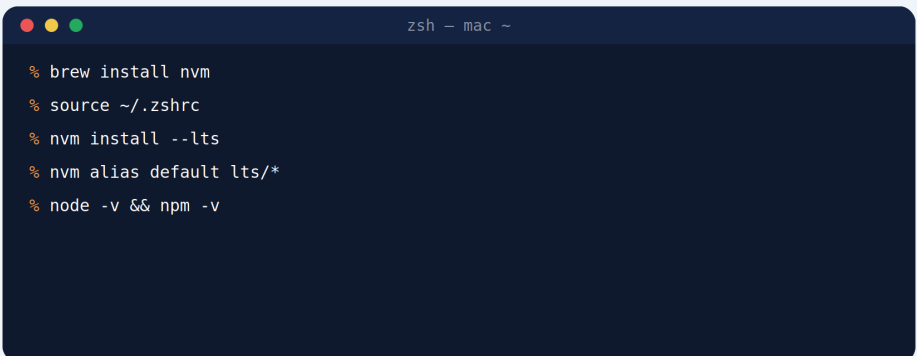
```
brew install nvm
mkdir -p ~/.nvm
```

2. Adicionar a configuração do nvm ao perfil do shell (zsh):

```
echo 'export NVM_DIR="$HOME/.nvm"' >> ~/.zshrc
echo '[ -s "/opt/homebrew/opt/nvm/nvm.sh" ] && . "/opt/homebrew/opt/nvm/nvm.sh"'
>> ~/.zshrc
source ~/.zshrc
```

3. Instalar e definir a versão LTS como padrão:

```
nvm install --lts
nvm use --lts
nvm alias default "lts/*"
```



```
zsh - mac ~
% brew install nvm
% source ~/.zshrc
% nvm install --lts
% nvm alias default lts/*
% node -v && npm -v
```

Figura 3 — Instalação do nvm e do Node.js LTS em ambiente macOS.

7 NPM E UTILITÁRIOS GLOBAIS

O curso padroniza o npm como gerenciador de pacotes (vem junto com o Node.js e é o mais usado no mercado).

```
npm -v  
npm config set init-author-name "Seu Nome"  
npm config set init-license "MIT"
```

Instalar globalmente as ferramentas de linha de comando usadas ao longo do curso:

```
npm install -g typescript ts-node tsx nodemon
```

- **typescript** — compilador TypeScript.
- **ts-node** — executa arquivos .ts diretamente, sem etapa manual de build.
- **tsx** — alternativa mais rápida ao ts-node, cada vez mais comum em projetos novos.
- **nodemon** — reinicia o processo automaticamente ao salvar arquivos.

Nota pedagógica

Instalar essas ferramentas de forma global é conveniente para estudo. No projeto guiado, elas também vão aparecer como devDependencies de cada projeto — depender apenas do que está instalado global na máquina do desenvolvedor é uma causa clássica do problema "na minha máquina funciona".

8 EDITOR DE CÓDIGO

Padrão do curso: Visual Studio Code (gratuito, multiplataforma e o mais usado em times Node.js/TypeScript hoje).

Extensões mínimas obrigatórias:

- **ESLint** — integra o linter ao editor, mostrando erros/avisos em tempo real.
- **Prettier** – Code formatter — formatação automática e consistente do código.
- **EditorConfig for VS Code** — respeita as regras de indentação/charset definidas no projeto.
- **Docker** — gerenciar containers e imagens direto do editor.
- **Thunder Client** ou **REST Client** — testar chamadas HTTP sem sair do editor.

- **GitLens** — visualizar histórico e autoria de cada linha de código.
- **DotENV** — realce de sintaxe para arquivos .env.

Instalação via linha de comando (após habilitar o comando "code" no PATH):

```
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode
code --install-extension editorconfig.editorconfig
code --install-extension ms-azuretools.vscode-docker
code --install-extension eamodio.gitlens
```

9 GIT E GITHUB

9.1 INSTALAÇÃO

- Windows 11: winget install --id Git.Git -e --source winget.
- Linux: sudo apt install git (Debian/Ubuntu) ou equivalente da distribuição.
- macOS: brew install git.

9.2 CONFIGURAÇÃO INICIAL

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
git config --global init.defaultBranch main
```

9.3 AUTENTICAÇÃO VIA SSH COM O GITHUB

Evite usar usuário/senha ou tokens digitados manualmente a cada envio de código. No Windows 11, o cliente OpenSSH já vem nativo no sistema — os mesmos comandos funcionam em PowerShell, Linux e macOS.

```
ssh-keygen -t ed25519 -C "seu-email@exemplo.com"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

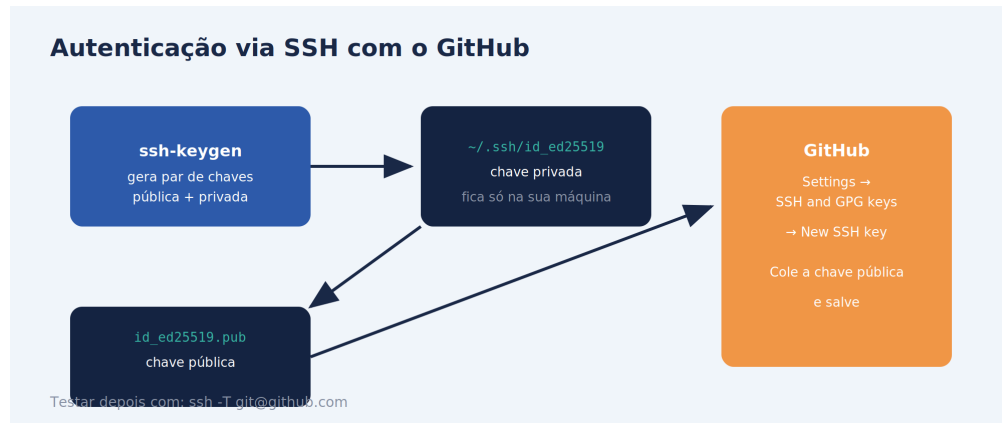


Figura 4 — Fluxo de autenticação SSH entre a máquina do aluno e o GitHub.

Copiar a saída do último comando e cadastrar em GitHub → Settings → SSH and GPG keys → New SSH key. Em seguida, testar:

```
ssh -T git@github.com
```

10 DOCKER E DOCKER COMPOSE

Desde a primeira semana, banco de dados, cache e outros serviços de apoio do projeto guiado sobem via Docker Compose — nunca instalados nativamente na máquina do aluno. Isso reproduz o fluxo real de qualquer time de desenvolvimento atual: o ambiente é descartável e reproduzível para qualquer pessoa da equipe.

10.1 WINDOWS 11: DOCKER DESKTOP COM BACKEND HYPER-V

Download disponível em docker.com/products/docker-desktop.

1. Habilitar os recursos nativos do Windows necessários, no PowerShell como Administrador:

```
dism.exe /online /enable-feature /featurename:Microsoft-Hyper-V /all /norestart
dism.exe /online /enable-feature /featurename:Containers /all /norestart
```

2. Reiniciar o computador quando solicitado.
3. Instalar o Docker Desktop e, na configuração inicial, selecionar o backend Hyper-V (não o WSL2) em Settings → General.

Esse caminho usa exclusivamente a virtualização nativa da Microsoft (Hyper-V), sem qualquer subsistema Linux adicional.

10.2 MACOS: DOCKER DESKTOP

No macOS, atenção à arquitetura do processador (Apple Silicon ou Intel) ao baixar o instalador correto em docker.com/products/docker-desktop.

10.3 LINUX: DOCKER ENGINE E COMPOSE

Seguir sempre o passo a passo oficial atualizado em docs.docker.com, pois as chaves e repositórios mudam periodicamente. Resumo do fluxo:

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER
```

10.4 VERIFICAÇÃO E PRIMEIRO COMPOSE DO CURSO

```
docker --version
docker compose version
docker run hello-world
```

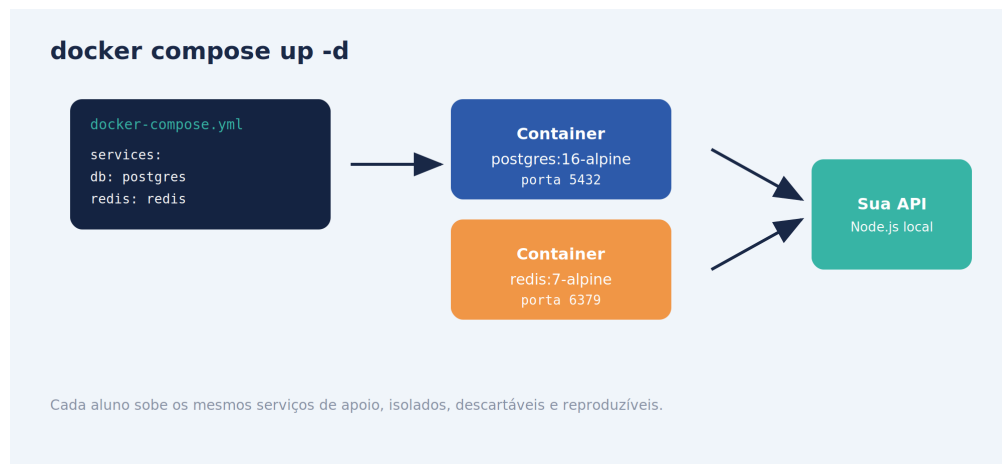


Figura 5 — Serviços de apoio subindo via Docker Compose.

11 CLIENTE HTTP E CLIENTE DE BANCO DE DADOS

Cliente HTTP recomendado: Insomnia — gratuito, leve, permite versionar as coleções de requisições junto com o repositório do projeto.

Alternativas aceitas: Postman ou a extensão Thunder Client direto no VS Code.

Cliente de banco de dados recomendado: DBeaver — gratuito, multiplataforma, suporta Postgres, MySQL e SQLite com a mesma interface.

12 QUALIDADE DE CÓDIGO: ESLINT, PRETTIER E EDITORCONFIG

Estas ferramentas entram como devDependencies em cada projeto — não apenas como configuração global do editor — porque a regra de lint e formatação precisa valer para qualquer pessoa que clonar o repositório, independente do editor que utilize.

```
npm install --save-dev eslint prettier eslint-config-prettier
```

Arquivo .editorconfig na raiz de cada projeto:

```
root = true
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 2
```

13 VALIDAÇÃO FINAL: HELLO WORLD PONTA A PONTA

Antes de encerrar o módulo, cada aluno valida o ambiente completo com um teste mínimo, primeiro local e depois em container.

13.1 EXECUÇÃO LOCAL

```
mkdir hello-node && cd hello-node

npm init -y

npm install --save-dev typescript tsx @types/node

npx tsc --init

mkdir -p src

echo 'console.log("Ambiente OK, " + process.version);' > src/index.ts

npx tsx src/index.ts
```

13.2 Execução em Container

Depois de executar o projeto diretamente no computador, vamos rodar a mesma aplicação dentro de um **container Docker**.

O objetivo é entender como padronizar o ambiente de execução.

Na execução local:

`npx tsx src/index.ts`

usamos o Node.js e as dependências instaladas na própria máquina.

Com Docker, a aplicação roda dentro de um ambiente isolado:

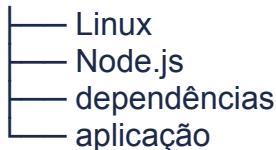
Computador



Docker



Container



Isso ajuda a reduzir diferenças entre Windows, macOS, Linux e servidores.

Criando o Dockerfile

Na raiz do projeto, crie um arquivo chamado:

Dockerfile

```
FROM node:24-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "tsx", "src/index.ts"]
```

Conteúdo:

Entendendo cada linha

FROM node:24-alpine

Utiliza uma imagem Linux que já possui Node.js e npm.

WORKDIR /app

Define /app como diretório principal dentro do container.

COPY package*.json ./

Copia o package.json e o package-lock.json.

RUN npm ci

Instala as dependências exatamente conforme o package-lock.json.
COPY ..

Copia o restante do projeto.
CMD ["npm", "run", "build", "src/index.ts"]

node:24-alpine

Define o comando que será executado quando o container iniciar.
Construindo a imagem

Execute:

```
% docker build -t hello-node .
```

Esse comando cria uma imagem chamada:
hello-node

Podemos verificar com:

```
sorteio@SestitoDev hello-node % docker images hello-node
REPOSITORY TAG IMAGE ID CREATED SIZE
hello-node latest fb275675d20a 3 minutes ago 275MB
```

Executando o container

Agora execute:
docker run --rm hello-node

Resultado esperado:

```
sorteio@SestitoDev hello-node % docker run --rm hello-node
Ambiente OK, v24.19.0
```

O parâmetro:
--rm

remove automaticamente o container quando sua execução terminar.

```
Fluxo
Dockerfile
↓
docker build
↓
Imagem hello-node
↓
docker run
↓
Container
↓
Aplicação Node.js
```

Conceito importante

Localmente:

```
npx tsx src/index.ts
```

Com Docker:

```
docker run --rm hello-node
```

O código é o mesmo. O que muda é o ambiente onde ele está sendo executado. Essa padronização é muito utilizada em desenvolvimento profissional, testes, CI/CD, homologação e produção.

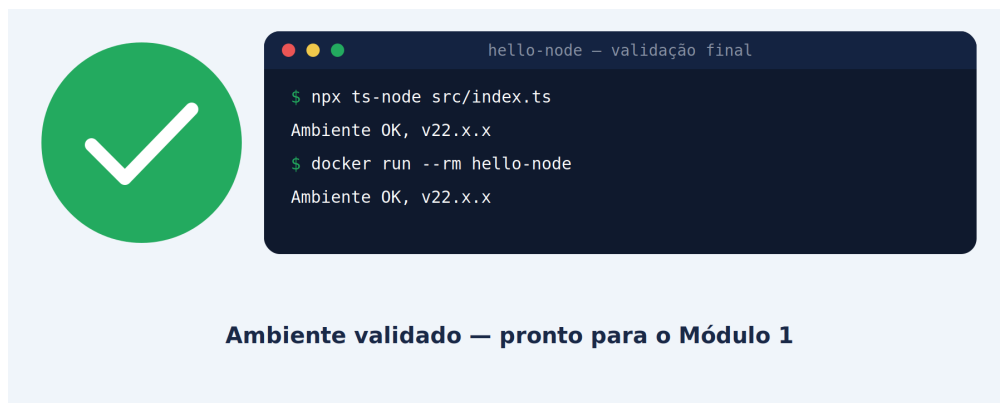


Figura 6 — Validação final do ambiente, local e em container.

Se as duas execuções imprimirem a mensagem sem erro, o ambiente está validado e a turma está pronta para o Módulo 1.

14 RECURSOS ADICIONAIS DO ECOSISTEMA NODE.JS

O projeto guiado deste curso usa uma stack enxuta o suficiente para caber em oito módulos, mas o ecossistema Node.js é muito mais amplo do que o que será efetivamente utilizado nas aulas. Esta seção apresenta, em caráter de referência, ferramentas e bibliotecas que qualquer desenvolvedor Node.js profissional deve conhecer — mesmo que este curso não as adote diretamente. O objetivo é que a turma saiba que essas opções existem, para que ganho vocacional servem e quando cada uma tende a ser escolhida no mercado.



14.1 FRAMEWORKS WEB (NESTJS, EXPRESS, FASTIFY)

Express é o framework web minimalista mais usado historicamente em Node.js — ótimo para aprender os fundamentos de rotas e middlewares, mas não impõe nenhuma estrutura de projeto.

Fastify segue a mesma filosofia minimalista do Express, mas com foco explícito em desempenho e validação de schema integrada (JSON Schema), sendo uma alternativa cada vez mais comum em APIs de alto tráfego.

NestJS é um framework opinativo, inspirado em Angular, que estrutura a aplicação em módulos, controllers, services e providers com injeção de dependência nativa. É a escolha mais comum em empresas que já têm um time grande e querem impor um padrão arquitetural consistente entre vários desenvolvedores — o equivalente, no mundo Node.js, ao que o Spring Boot representa no mundo Java.

14.2 ACESSO A DADOS (PRISMA, TYPEORM, DRIZZLE)

Prisma é o ORM mais adotado atualmente em projetos TypeScript novos: gera um cliente totalmente tipado a partir de um schema declarativo, com migrations versionadas. TypeORM segue um estilo mais próximo de Hibernate/JPA, com decorators e Active Record ou Data Mapper. Drizzle é um ORM mais recente, mais leve e mais próximo do SQL puro, ganhando espaço em projetos que priorizam performance e previsibilidade das queries geradas.

14.3 VALIDAÇÃO DE DADOS (ZOD, CLASS-VALIDATOR)

Zod permite declarar schemas de validação e derivar automaticamente os tipos TypeScript correspondentes, eliminando duplicidade entre validação e tipagem. class-validator usa decorators sobre classes e é a opção mais comum dentro de projetos NestJS, por se integrar naturalmente ao seu sistema de DTOs.

14.4 TESTES AUTOMATIZADOS (JEST, VITEST, SUPERTEST)

Jest é o framework de testes mais tradicional do ecossistema JavaScript/TypeScript, com mocks, cobertura e snapshots integrados. Vitest é uma alternativa mais recente, compatível com a API do Jest, mas mais rápida em projetos que já usam Vite/ESM. Supertest complementa qualquer um dos dois para testes de integração de APIs HTTP, simulando requisições reais contra a aplicação.

14.5 OBSERVABILIDADE (PINO, WINSTON)

Pino é uma biblioteca de logging estruturado (JSON) focada em desempenho, muito usada em APIs de alto volume. Winston é mais flexível em termos de formatos e destinos de log (arquivo, console, serviços externos), sendo uma escolha comum quando o projeto precisa de múltiplos transports configurados.

14.6 PROCESSOS EM PRODUÇÃO (PM2)

PM2 é um gerenciador de processos para aplicações Node.js em produção: reinicia o processo automaticamente em caso de falha, permite rodar múltiplas instâncias em modo cluster e centraliza logs. É comum em servidores que não usam orquestradores como Kubernetes.

14.7 DOCUMENTAÇÃO DE API (SWAGGER/OPENAPI)

A especificação OpenAPI (Swagger) descreve os endpoints de uma API de forma padronizada e gera automaticamente documentação interativa. Frameworks como NestJS têm suporte nativo para gerar essa documentação diretamente a partir dos decorators do código, mantendo a documentação sempre sincronizada com a implementação.

14.8 FILAS E PROCESSAMENTO ASSÍNCRONO (BULLMQ)

BullMQ é uma biblioteca de filas baseada em Redis, usada para processar tarefas de forma assíncrona e desacoplada da requisição HTTP original — como envio de e-mails, geração de

relatórios ou processamento de pagamentos. É a peça que normalmente falta quando um sistema começa a sofrer com timeouts em operações demoradas.

14.9 AUTENTICAÇÃO E AUTORIZAÇÃO (PASSPORT.JS, JWT)

Passport.js é um middleware de autenticação com dezenas de "estratégias" prontas (login local, OAuth com Google/GitHub, etc.). A biblioteca jsonwebtoken é normalmente usada em conjunto, para emitir e validar tokens JWT entre o backend e o cliente.

14.10 SEGURANÇA (HELMET, RATE LIMITING)

Helmet configura automaticamente cabeçalhos HTTP de segurança recomendados (proteção contra clickjacking, sniffing de MIME type, entre outros). Bibliotecas de rate limiting (como express-rate-limit) protegem a API contra abuso e ataques de força bruta, limitando o número de requisições por IP em um intervalo de tempo.

14.11 AUTOMAÇÃO DE QUALIDADE (HUSKY, LINT-STAGED, COMMITLINT)

Husky permite configurar hooks do Git (como pre-commit e pre-push) para rodar scripts automaticamente. Combinado com lint-staged, garante que apenas os arquivos alterados sejam validados pelo ESLint/Prettier antes de cada commit. Commitlint valida se as mensagens de commit seguem um padrão (como Conventional Commits), o que facilita a geração automática de changelogs.

14.12 MONOREPOSITÓRIOS (NX, TURBOREPO, PNPM WORKSPACES)

Quando um projeto cresce para múltiplos pacotes (por exemplo, uma API e um painel administrativo compartilhando código), ferramentas de monorepo como Nx e Turborepo organizam builds, testes e cache entre pacotes de forma eficiente. pnpm workspaces oferece uma solução mais simples, nativa do próprio gerenciador de pacotes pnpm, para o mesmo problema.

Como usar esta seção

Não é necessário memorizar todas as ferramentas listadas aqui. O objetivo é reconhecer o nome, entender em qual categoria do dia a dia profissional cada uma resolve um problema, e saber onde procurar quando esse problema aparecer em um projeto real.

15 SOLUÇÃO DE PROBLEMAS

15.1 WINDOWS

- Comando "code" não reconhecido no terminal: re-instalar o VS Code marcando "Add to PATH".
- Docker Desktop não inicia: verificar se a virtualização está habilitada na BIOS/UEFI e se os recursos Hyper-V e Containers foram ativados (Painel de Controle → Programas → Ativar ou desativar recursos do Windows).
- Permissão negada em scripts PowerShell: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser.
- winget não encontrado: atualizar o aplicativo "App Installer" pela Microsoft Store.

15.2 LINUX

- "docker: permission denied": fazer logout/login após adicionar o usuário ao grupo docker.
- Porta 5432 ou 6379 já em uso: verificar se já existe um Postgres/Redis instalado nativamente.

15.3 MACOS

- Ferramentas de linha de comando ausentes: executar xcode-select --install.
- Homebrew não encontrado no PATH em Apple Silicon: confirmar o eval do brew shellenv no ~/.zshrc.

16 CONSIDERAÇÕES FINAIS

Este módulo não tem como objetivo apresentar conceitos novos de programação, e sim eliminar, já na primeira semana, o atrito mais comum de alunos com ambientes diferentes, versões divergentes e ferramentas ausentes. Ao padronizar o ambiente antes do primeiro commit do projeto guiado, o tempo das próximas aulas fica inteiramente disponível para arquitetura, código e discussão técnica.

O próximo material (Módulo 1) parte deste ambiente já validado para iniciar a arquitetura do projeto guiado que acompanhará a turma pelas próximas semanas do curso.

17 ATIVIDADES DE FIXAÇÃO

As duas atividades a seguir fecham o Módulo 0. A primeira é individual e teórica, com entrega escrita; a segunda é prática, com entrega em repositório Git. Ambas devem ser realizadas com o ambiente já validado na Seção 13.

17.1 ATIVIDADE TEÓRICA — PESQUISA E FUNDAMENTAÇÃO

Objetivo: pesquisar e descrever, com as próprias palavras, conceitos fundamentais de Node.js e TypeScript que sustentam todo o restante do curso.

Cada aluno deve entregar um documento individual, em formato de texto corrido (sem colar trechos de documentação sem citação), respondendo aos itens a seguir:

1. O que é o event loop do Node.js e por que ele permite atender muitas conexões simultâneas com uma única thread principal.
2. Qual é a diferença entre operações síncronas e assíncronas em JavaScript, e por que bloquear o event loop é considerado um erro grave em produção.
3. O que é TypeScript e qual problema ele resolve em relação ao JavaScript puro (explicar a diferença entre tipagem estática em tempo de compilação e tipagem dinâmica em tempo de execução).
4. Qual é a diferença entre CommonJS (require/module.exports) e ES Modules (import/export) no Node.js, e por que essa diferença ainda gera problemas de compatibilidade em projetos reais.
5. Pesquisar e comparar, em pelo menos um parágrafo cada, três frameworks web citados na Seção 14.1 (Express, Fastify e NestJS), indicando um cenário real em que cada um seria a escolha mais indicada.
6. O que é um gerenciador de versões do Node.js (nvm) e por que ele é considerado prática obrigatória em times profissionais, conforme discutido na Seção 3.
7. O que é um container Docker e qual é a diferença conceitual entre um container e uma máquina virtual.

CrITÉrios de avaliação: clareza da explicação, uso correto da terminologia técnica, presença de exemplo próprio (não copiado da documentação) e citação das fontes pesquisadas em formato ABNT.

Entrega: documento de texto (Word ou PDF), de 2 a 4 páginas, enviado individualmente até a data definida em aula pelo professor.

17.2 ATIVIDADE PRÁTICA — HELLO-NODE PROFISSIONAL

Objetivo: evoluir o projeto "hello-node" da Seção 13 para um pequeno projeto TypeScript organizado, versionado no GitHub e com qualidade de código configurada — replicando, em escala reduzida, o fluxo profissional que será usado no projeto guiado.

A partir do projeto hello-node já criado, cada aluno deve:

1. Criar um repositório na Organização do GitHub (privado ou público) e conectar o projeto local a ele via SSH, usando a configuração da Seção 9.3.
2. Configurar o tsconfig.json em modo estrito ("strict": true) e corrigir eventuais erros de tipagem que aparecerem.
3. Instalar e configurar o ESLint e o Prettier como devDependencies do projeto (Seção 12), com o arquivo .editorconfig na raiz.
4. Adicionar ao menos uma dependência externa via npm (por exemplo, a biblioteca "dayjs" para datas, ou a "zod" apresentada na Seção 14.3) e utilizá-la em uma função simples dentro de src/index.ts.
5. Escrever, no arquivo README.md do repositório, uma breve explicação (mínimo 3 parágrafos) descrevendo: o que o projeto faz, quais decisões técnicas foram tomadas (por exemplo, por que aquela dependência foi escolhida) e como rodar o projeto localmente.
6. Publicar o projeto com pelo menos três commits com mensagens claras, mostrando a evolução do trabalho (não é aceito um único commit final com tudo pronto).

Critérios de aceite: repositório acessível no GitHub; projeto roda localmente com "npx ts-node src/index.ts" sem erros; ESLint/Prettier configurados e sem violações; README completo; histórico de commits coerente com o progresso do trabalho.

Entrega: link do repositório GitHub, enviado individualmente até a data definida em aula pelo professor.

REFERÊNCIAS

DOCKER INC. Docker documentation. Disponível em: <https://docs.docker.com>. Acesso em: 20 ago. 2026.

GITHUB. Connecting to GitHub with SSH. Disponível em: <https://docs.github.com>. Acesso em: 20 ago. 2026.

MICROSOFT. PowerShell documentation. Disponível em: <https://learn.microsoft.com/powershell>. Acesso em: 20 ago. 2026.

MICROSOFT. Windows Package Manager (winget) documentation. Disponível em: <https://learn.microsoft.com/windows/package-manager>. Acesso em: 20 ago. 2026.

MICROSOFT. Visual Studio Code documentation. Disponível em: <https://code.visualstudio.com/docs>. Acesso em: 20 ago. 2026.

NESTJS. NestJS documentation. Disponível em: <https://docs.nestjs.com>. Acesso em: 20 ago. 2026.

NODE.JS FOUNDATION. Node.js documentation. Disponível em: <https://nodejs.org/en/docs>. Acesso em: 20 ago. 2026.

NVM-SH. nvm — Node Version Manager. Disponível em: <https://github.com/nvm-sh/nvm>. Acesso em: 20 ago. 2026.

NVM-WINDOWS. nvm-windows documentation. Disponível em: <https://github.com/coreybutler/nvm-windows>. Acesso em: 20 ago. 2026.

PRISMA. Prisma documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 20 ago. 2026.

PRETTIER. Prettier documentation. Disponível em: <https://prettier.io/docs>. Acesso em: 20 ago. 2026.